WMS System Writeup

Comprehensive business and technical overview for panel presentation

1. Executive Summary

WMS is an in-house developed production and transfer management platform supporting slaughter operations, butchery processing, highcare workflows, despatch, QA, spices/chopping, sausage, beef/lamb, and related internal stock and asset movements. Built on open-source architecture, it is easy to tweak and extend for changing operational needs. The system combines real-time scale capture, transaction controls, IDT transfers, reporting/exports, and API-based data exchange.

1.1 Core Technology Stack

- Backend: PHP with Laravel framework.
- Frontend: Vanilla JavaScript with jQuery in Blade-rendered UI flows.
- Database: Structured relational data model supporting transactional and reporting workloads.

2. Architecture Snapshot

Application Layer

- Laravel 8 web application with Blade views and controller-driven workflows.
- Domain controllers per operational area (slaughter, butchery, despatch, highcare, etc.).
- Helper-driven integrations for scale reads, queue publishing, and utility formatting.

Data and Integration Layer

- Relational DB-backed operational tables (weights, transfers, stock, receipt lines).
- HTTP API exposure for downstream consumers and upstream imports.
- Excel/CSV export pipeline for reports and NAV-oriented outputs.

3. Major Functional Components

Component	Primary Function	Key Capabilities
Authentication and User Management	Access control and user session management	LDAP/domain authentication API login, local user provisioning, logout/session controls, user updates, permission editing, role-

Component	Primary Function	Key Capabilities
		driven behavior on screens
Slaughter	Inbound livestock weighing and classification	Scale read, net and settlement computation, grading/classification, missing slap handling, disease flags, SMS triggers, slaughter exports
Butchery	Processing operations across multiple scale stages	Scale 1/2/3 workflows, production process mapping, marination, splitting, beheading/breaking/deboning reports and exports
Highcare 1	IDT issue/receive/bulk/slicing workflows	Scale selection persistence, host-aware scale reads, IDT reporting, highcare-specific transfer logic
Sausage	IDT, production entries, stuffing flow	IDT create/receive, rights validation, item details APIs, per-batch and entries reporting
Spices and Chopping	Batch production and line-level execution	Template import, batch lifecycle, chopping v1/v2, posted lines reports and exports
Despatch	Dispatch-side transfer issue/receive and stock control	IDT issue and receive, variance and chiller views, stock take/import, history and summary exports
Generic IDT Module	Cross-department transfer orchestration	Issue, receive, approval queues, location-based filtering, historical reporting, summary exports
QA and Beef/Lamb	Specialized transfer and processing flows	QA issue/receive/reporting and beef/lamb slicing/receiving history
Assets and Stocks	Non-production movement and stock visibility	Asset movement lifecycle and stock transactions dashboard

4. API Integrations and Interfaces

4.1 Internal Web-AJAX Endpoints

- Slaughter weigh AJAX endpoints for receipt and vendor reconciliation before posting.
- Scale read and COM-port listing endpoints used by weigh screens.
- IDT and module-specific AJAX for product lookup, rights checks, and transfer metadata.

4.2 Public/External API Surface

- Token-protected and throttled endpoints for data extraction: `/api/v1/fetch-slaughter-data`, `/api/v1/fetch-missing-slaps`, `/api/v1/fetch-beheading-data`, `/api/v1/fetch-breaking-data`, `/api/v1/fetch-deboning-data`, `/api/v1/fetch-chopping-data`.
- Inbound ingestion endpoints for slaughter receipts and line pushes.
- Auxiliary API endpoints for barcode inserts and last-insert checks.

4.3 External Service Integrations

- LDAP/domain authentication API integration for enterprise sign-in (domain user credential validation).
- Scale API integration via configurable host/endpoints and COM-port routing.

- RabbitMQ integration capability for transfer/production event publishing.
- SMS service integration via configurable sender, API key, and destination settings.
- Mail service connectors (Mailgun/Postmark/SES) available in service configuration.

5. Scale Configuration and Weighing Controls

5.1 Scale Configuration Model

- Scale definitions are stored in scale configuration records keyed by section/workflow.
- Core attributes include scale name, COM port, tare weight, IP address, and optional baud rate.
- Scale settings are editable from dedicated module settings screens.

5.2 Runtime Scale Read Behavior

- Screens invoke API-based reads using selected COM-port + host endpoint.
- Highcare helper resolves blank IP addresses to localhost fallback for resiliency.
- UI feedback includes host badge visibility, reading timeout handling, and friendly error states.

5.3 Caching and Refresh

- Scale configs are cached for performance.
- Config update flows flush cache to guarantee fresh read behavior after edits.

6. IDT Transfer Lifecycle (Core Cross-Module Process)

6.1 Standard Lifecycle

1. Issue transfer from a source location with product, pieces/crates, and weight.
2. Route transfer to destination location and chiller where applicable.
3. Receive transfer with receiver totals and variance indication.
4. Apply approval flow for transfers flagged as requires_approval.
5. Track outcomes in history, summary reports, and export outputs.

6.2 Location Mapping and Governance

- IDT defines explicit operational location map (e.g., Butchery, Curing, Highcare, Sausage, Despatch, QA).
- Queries and UI filters use from/to locations to enforce contextual visibility.
- Special handling exists for specific despatch-linked location groups and approval queues.

6.3 Reporting and Export

- IDT history export produces line-level transfer outputs over selected date windows.
- IDT summary export aggregates sent vs received weight and pieces by product.
- Department-specific IDT reports (Despatch, Highcare, QA, Sausage, Freshcuts) support operational accountability.

7. Data Flows and Operational Reporting

7.1 Inbound Data Flows

- Receipt imports (including API-driven inserts) feed slaughter pre-weigh context.
- Master data (items/products/chillers/templates) drives validation and dropdown controls.

7.2 Processing Data Flows

- Scale reading and tare calculations produce net and settlement weights.
- Classification logic maps carcass type + settlement + meat percent to final class codes.
- IDT workflows propagate stock movement between operational locations.

7.3 Outbound Data Flows

- Excel and CSV exports support downstream finance/ERP and audit use cases.
- API fetch endpoints provide controlled access for external analytics/consumer systems.
- Queue publishing hooks support event-driven integration patterns where enabled.

8. Security, Controls, and Reliability

- Web features are primarily protected by authentication middleware and LDAP/domain-authenticated login flow.
- Public API extraction endpoints are protected via token middleware and request throttling.
- Server-side validation is used in key save/update paths and API date-range requests.
- Caching is used for heavy lookups; cache invalidation is included in settings updates.
- Operational logs are available through an integrated log viewer endpoint.

9. Key Technical Dependencies

- Laravel 8 and PHP 7.3/8.0 compatibility.
- Maatwebsite Excel for export pipelines.
- Guzzle HTTP client for external HTTP integrations.
- php-amqplib for RabbitMQ messaging capability.
- Toastr notifications and Laravel Log Viewer for operational UX and supportability.

10. Suggested Future Enhancements

- Consolidate integration configuration into an environment matrix by module and site.
- Expand API observability with endpoint-level monitoring and error budgets.

- Introduce workflow-level audit trails for approval and override events.
- Add standardized integration contracts for downstream ERP/NAV handoffs.
- Document SLA/SLO targets for scale read latency and transfer posting turnaround.

End of writeup.